DSA Assignment

Stack using Array & Circular Queue using Array

Language: C++

Q1. Design and Implement a Stack Using an Array

Operations: PUSH(x), POP(), PEEK(), DISPLAY()

Stack Overflow: It occurs when PUSH is performed on a full fixed-size stack.

Stack Underflow: It occurs when POP or PEEK is performed on an empty stack.

C++ Program

```
#include <iostream>
using namespace std;

class Stack {
    int arr[5];
    int top;
    const int capacity = 5;

public:
    Stack() {
        top = -1;
    }

    void PUSH(int x) {
        if (top == capacity - 1) {
            cout << "Stack Overflow!" << endl;
            return;
        }
        arr[++top] = x;
        cout << x << " pushed into stack." << endl;
    }

    void POP() {
        if (top == -1) {
            cout << "Stack Underflow!" << endl;
            return;
        }
        cout << arr[top--] << " popped from stack." << endl;
    }

    void PEEK() {
        if (top == -1) {
            cout << "Stack is empty." << endl;
            return;
        }
        cout << "Top element: " << arr[top] << endl;
    }

    void DISPLAY() {
        if (top == -1) {
            cout << "Stack is empty." << endl;
            return;
        }

        cout << "Stack elements: ";
        for (int i = top; i >= 0; i--)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    Stack s;

    s.PUSH(10);
    s.PUSH(20);
    s.PUSH(30);
    s.DISPLAY();
    s.PEEK();
```

```
    s.POP();
    s.DISPLAY();

    // Uncomment to test overflow:
    // s.PUSH(40); s.PUSH(50); s.PUSH(60);
    // s.PUSH(70); s.PUSH(80); s.PUSH(90);

    // Uncomment to test underflow:
    // s.POP(); s.POP(); s.POP(); s.POP(); s.POP(); s.POP();

    return 0;
}
```

Q1. Theory / Complexity Analysis

Operation	Time Complexity	Space Complexity
PUSH(x)	O(1)	O(1) extra
POP()	O(1)	O(1) extra
PEEK()	O(1)	O(1) extra
DISPLAY()	O(n)	O(1) extra

Overall space complexity: O(n), because the array stores at most n elements. Each individual operation uses O(1) extra space except DISPLAY, which also uses only O(1) auxiliary space.

Fixed-size stack: If the stack capacity is fixed and the user tries to insert more elements than the capacity, Stack Overflow occurs. The new element is not inserted and the program should display an overflow message.

Q2. Implement a Circular Queue Using an Array

Operations: ENQUEUE(x), DEQUEUE(), FRONT(), DISPLAY()

The circular queue uses modulo arithmetic so that REAR and FRONT can wrap around to the beginning of the array.

Full condition: (rear + 1) % capacity == front when the queue is using one empty slot to distinguish full from empty.

Empty condition: front == -1.

C++ Program

```
#include <iostream>
using namespace std;

class CircularQueue {
    int arr[5];
    int front, rear;
    const int capacity = 5;

public:
    CircularQueue() {
        front = -1;
        rear = -1;
    }

    bool isEmpty() {
        return front == -1;
    }

    bool isFull() {
        return (rear + 1) % capacity == front;
    }

    void ENQUEUE(int x) {
        if (isFull()) {
            cout << "Queue is Full!" << endl;
            return;
        }

        if (isEmpty()) {
            front = rear = 0;
        } else {
            rear = (rear + 1) % capacity;
        }

        arr[rear] = x;
        cout << x << " inserted into queue." << endl;
    }

    void DEQUEUE() {
        if (isEmpty()) {
            cout << "Queue is Empty!" << endl;
            return;
        }

        cout << arr[front] << " deleted from queue." << endl;

        if (front == rear) {
            front = rear = -1;
        } else {
            front = (front + 1) % capacity;
        }
    }

    void FRONT() {
        if (isEmpty()) {
            cout << "Queue is Empty!" << endl;
            return;
        }
        cout << "Front element: " << arr[front] << endl;
    }

    void DISPLAY() {
        if (isEmpty()) {
            cout << "Queue is Empty!" << endl;
            return;
        }

        cout << "Queue elements: ";
```

```
        int i = front;

        while (true) {
            cout << arr[i] << " ";
            if (i == rear)
                break;
            i = (i + 1) % capacity;
        }
        cout << endl;
    }
};

int main() {
    CircularQueue q;

    q.ENQUEUE(10);
    q.ENQUEUE(20);
    q.ENQUEUE(30);
    q.DISPLAY();
    q.FRONT();

    q.DEQUEUE();
    q.DISPLAY();

    q.ENQUEUE(40);
    q.ENQUEUE(50);
    q.ENQUEUE(60);
    q.DISPLAY();

    // Uncomment to test full condition:
    // q.ENQUEUE(70);

    // Uncomment to test empty condition:
    // q.DEQUEUE(); q.DEQUEUE(); q.DEQUEUE();
    // q.DEQUEUE(); q.DEQUEUE(); q.DEQUEUE();

    return 0;
}
```

Q2. Theory / Comparison with Simple Linear Queue

1. Better memory utilization: In a circular queue, when elements are removed from the front, the freed positions at the beginning can be reused. The rear wraps around to these positions. Therefore, memory is utilized efficiently.

2. Time complexity: ENQUEUE and DEQUEUE both take O(1) time because insertion and deletion are performed directly using front/rear indices. FRONT also takes O(1). DISPLAY takes O(n).

3. Space complexity: The queue uses a fixed array of capacity n, so its space complexity is O(n).

4. Problem in a linear queue: When REAR reaches the last index, a simple linear queue may report overflow even if there are unused positions at the beginning created by previous DEQUEUE operations. This is called **false overflow** or **space wastage**. A circular queue solves this by wrapping REAR back to the beginning.

Quick Comparison

Feature	Linear Queue	Circular Queue
Memory reuse	Poor	Better
ENQUEUE	O(1) until rear reaches end	O(1)
DEQUEUE	O(1)	O(1)
Space	O(n)	O(n)
Unused beginning positions	May remain unused	Reused

Conclusion: An array-based stack follows the LIFO (Last In, First Out) principle, while a circular queue follows FIFO (First In, First Out). Both provide efficient O(1) insertion/deletion operations when implemented with appropriate indices.